

An implementation of Open-RMF in an industrial environment

Michele Belletti

2025
December

Contents

I	Introduction	2
1	Background and motivation	4
2	Research questions and objectives	5
3	Scope and limitations	6
II	Literature Review	7
4	Overview of ROS1 and ROS2	9
4.1	Structure of ROS	10
4.2	Tools of ROS	11
5	Introduction to NAV2 and Google Cartographer	12
5.1	Description of Nav2	12
5.2	Cartographer	12
6	Review of related work on porting navigation stack from ROS1 to ROS2	13
7	Overview of open source fleet management systems, particularly Open-RMF	14
7.1	Open-RMF description	14

Part I

Introduction

This work is a study on a new system OPEN-RMF that is still in active development but has reached a good state to apply for the robots in the facility Elettra Sincrotrone Trieste

Chapter 1

Background and motivation

Elettra Sincrotrone Trieste¹ is a big research facility that, principally, gives to users two light sources: the synchrotron Elettra and the laser at free electron Fermi. Besides this, it study new technology and technique to improve the their capabilities and work activities. A lot of the latest improvements are based on robots, that also after the covid and latest with the exploding diffusion of LLMs, allow an interaction of the users of the light sources to communicates and interact with the robots in the facility to obtain information and services as transport of packages. They already developed a global planner D-Star (D-star based optimized trajectory planner) [2] in ROS 1 for sending a robot in the infrastructure that is optimized for exploring a big space. But ROS 1 has reached the end of life so we need to rewrite the code using the ROS 2 ecosystem. Furthermore, because there are multiple vendor that are specialized for different type of work, there is the problem of the interaction between them. For this in the latest years is born Open-RMF² a multi platform for control and manage a multi fleet float of robots.

¹<https://www.elettra.eu/>

²<https://www.open-rmf.org/>

Chapter 2

Research questions and objectives

Our objects for this thesis is: given the company Mini RoverRobotics¹ Robot



Figure 2.1: Rover Mini of the RoverRobotics

import it to ROS 2 Nav2 ecosystem using a porting of the global planner D-star from ROS 1 to ROS 2[1] and then add it to Open-RMF to control it. Compare it to the ROS 1 stack and validate the use of the Open-RMF ecosystem in Elettra Sincrotrone Trieste.

¹<https://roverrobotics.com/>

Chapter 3

Scope and limitations

The principal limitation is the now non accessible control for the lifts that will not enable us to have a direct control on them and also there is not an automatic open-close control over the doors. Even if the high customability of open-rmf is possible to use a human interaction for active these systems, like when the robot reach a door, it the send a message or a light can be turned on and then wait until someone will open the door or a lift. But is not a good approach.

Part II

Literature Review

In this part we will provide a description of the technologies used in this thesis. We will start describing what is ROS and how it works then we will talk about the navigation system used and how it is implemented the control with the fleet management system Open-RMF

Chapter 4

Overview of ROS1 and ROS2

The Robotics Operating System ¹ is a compilation of tools and libraries open-source that help the development of application in the robotic environment. otherwise from the name, it's not a Operating System (OS) but an Software Development Kit (SDK) that gives at the user all the tools for creating your robot giving some typically functionality of an OS: hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management. The ROS ecosystem is based on (all these things is going to be described in detail in the next section):

- An communication system named middleware or plumbing. This allow the Nodes to share information between each other with an anonymous publish/subscribe pattern allowing fault isolation, separation of concerns and clear interfaces and a synchronous system RPC-style communication between services.
- Tools to control and improve the development of the software like launch, introspection, debugging, visualization, plotting, logging, and playback.
- Provides tools and libraries for obtaining, building, writing, and running code across multiple computers.
- A active and big community with at the center the Open Robotics².

¹<https://www.ros.org/>

²<https://www.openrobotics.org/>

4.1 Structure of ROS

The documentation³ give all the information of ROS, some functionality write below are not available for ROS 1:

- The **ROS Graph** is a peer-to-peer network of processes that elaborated simultaneously and communicate with each other.
- **Nodes** is a modular process and should be a specific resolution for a problem to be more easily communicate with other Nodes through topics, services, action and parameters.
- **Topics** are the bus for communication between Nodes to share a streams of messages in a publisher to subscriber structure.

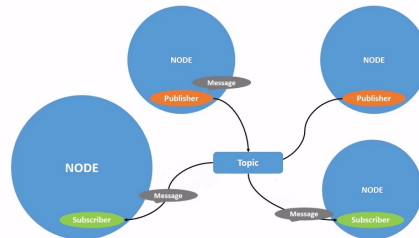


Figure 4.1: Topic process

- **Service** are based on the request and response model. When a node with a Service client send a Request Message to Node with a Service Server it will send a Response Message to that specific Service Client.

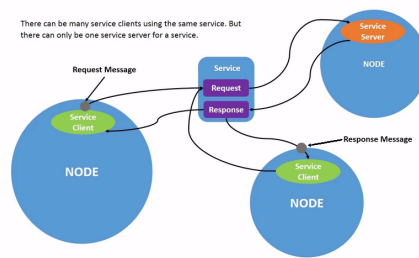


Figure 4.2: Service process

- **Parameters** are configuration of a node and allow to modified them without changing the code.

³<https://docs.ros.org/en/rolling/index.html>

- **Actions** were created to fulfill the needs of something for tracking long task. They are a combination of two service (goal and result) and a pub-sub (feedback). It send a goal request and receive a response then send a result request, the node then start to pub to the feedback topic and then response to the result request.

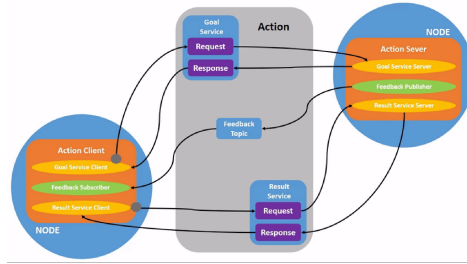


Figure 4.3: Action process

4.2 Tools of ROS

Now i will describe the tools that gives ROS to develop robotics system:

- **Launch** is a file structure that allow to start multiple nodes with a set of parameters.
- **rqt** is a graphical user interface (GUI) tool for ROS 2. With a series of plugins allow to interact with an interface for helping in the development, in the monitoring and diagnostic the ROS system.
- **rqt_console** visualize the log generate by the ROS 2 system.
- **Rviz** is a 3D visualizer for the Robot Operating System framework?

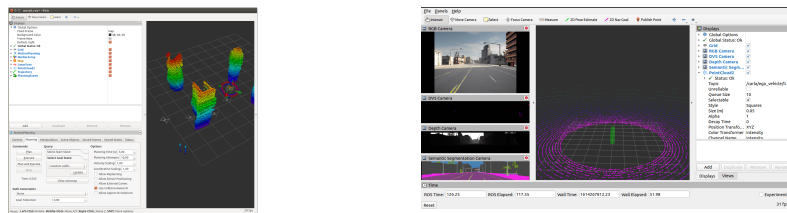


Figure 4.5: Examples of visualization of ROS data with Rviz

- **rostopic** .

Chapter 5

Introduction to NAV2 and Google Cartographer

5.1 Description of Nav2

5.2 Cartographer

Chapter 6

Review of related work on porting navigation stack from ROS1 to ROS2

Chapter 7

Overview of open source fleet management systems, particularly Open-RMF

7.1 Open-RMF description

...

Bibliography

- [1] Steven Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [2] Andrey Vukolov et al. “Universal Configurable Navigation and Control System for Industrial Unmanned Ground Vehicles with Differential Chassis”. In: *Advances in Mechanism and Machine Science*. Ed. by Masafumi Okada. Cham: Springer Nature Switzerland, 2023, pp. 287–301. ISBN: 978-3-031-45770-8.